Reqly

Technical Overview

Architecture · Installation · Tech stack · Database schema · Roadmap

Version	1.0.0
Document date	23 October 2026
Repository	github.com/ShaikRaihan93/reqly
Author	Shaik Raihan

The API client that stays on your machine.

1. Overview

Reqly is a local, Postman-style API client. It runs entirely on the user's machine: a Spring Boot server bound to 127.0.0.1 serves a single-page web UI, stores everything in an embedded H2 database, and executes HTTP requests on the user's behalf. Nothing is sent to a cloud service. Each local account has its own private workspace.

Key capabilities

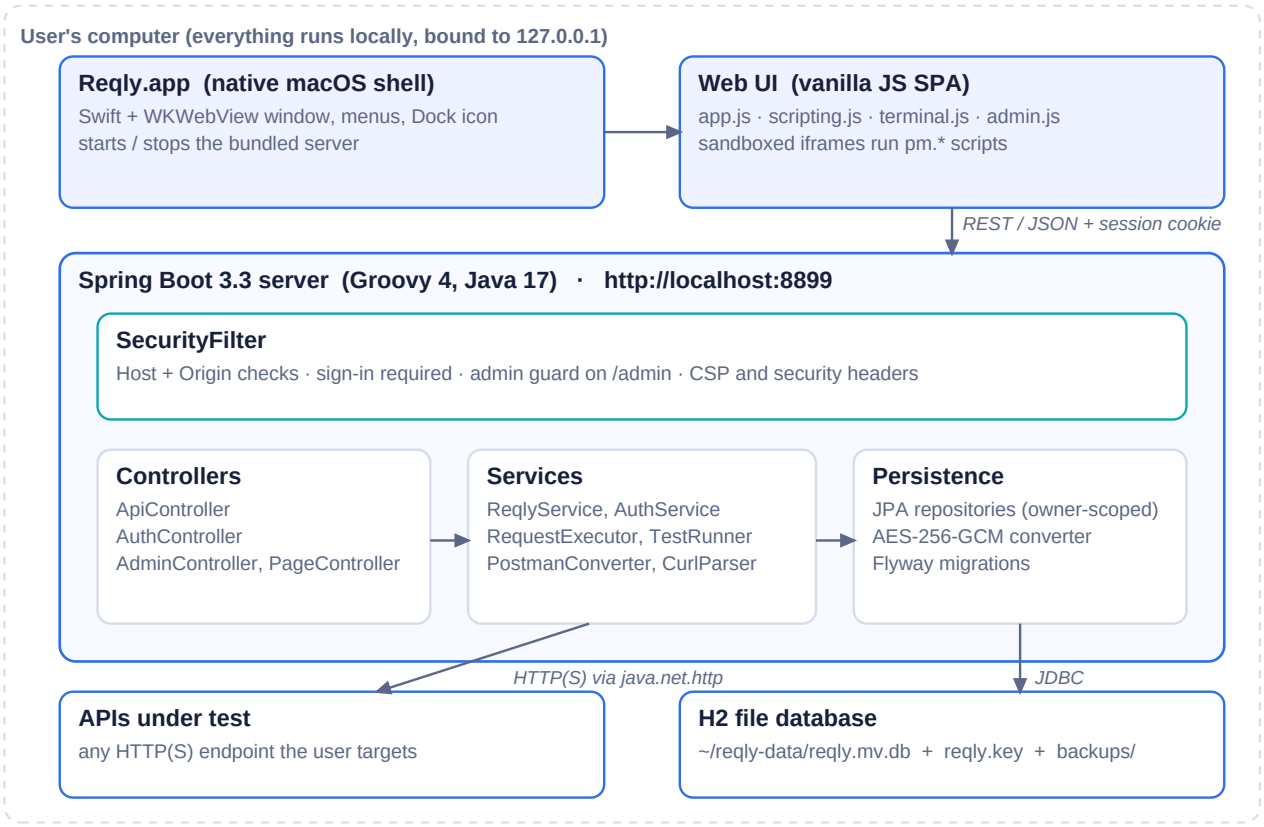
- Collections with nested folders, drag-and-drop ordering, duplicate / move / rename, and request tabs with unsaved-change tracking.
- All common body types (JSON, text, x-www-form-urlencoded, multipart form-data with files, GraphQL), Bearer / Basic / API-key auth, per-request timeout and redirect settings.
- Environments with secret variables, {{variable}} highlighting and hover previews, value capture from responses for request chaining.
- One-line assertions, Postman-style pre-request and post-response scripts (pm API, console.log) in a sandboxed iframe.
- Response viewer: lossless JSON tree, JSONPath filter, in-body search, HTML preview, per-request response history with diff.
- Collection runner with iterations, CSV/JSON data files, parallelism and exportable reports; cURL and Postman import/export; full backup/restore.
- Command terminal and console dock, Cmd+K palette, three themes (sci-fi, dark, light), keyboard shortcuts, accessibility support.
- Accounts with sign-up / sign-in, and an admin dashboard for users, tables, read-only SQL and backups.

Version information

Item	Value
Product version	1.0.0 (Gradle `version = '1.0.0'`)
Database schema version	Flyway V3 (V1 baseline, V2 accounts, V3 admin role)
Runtime	Java 17, bundled in the macOS app via jpackage
Default address	`http://localhost:8899` (loopback only)
Data directory	`~/reqly-data/` (database, encryption key, backups, logs)
Supported OS (installer)	macOS 12 or later; runs from source on any OS with Java 17
Source	github.com/ShaikRaihan93/reqly, branch main

2. Architecture

Reqly is a three-part local application: a native window, a web front end, and a Spring Boot back end with an embedded database. All components run on one machine and talk over the loopback interface.



Components

Layer	Responsibility
Native shell	Reqly.app's executable (Swift, WKWebView). Starts the bundled Java server, waits until it answers, shows the UI in its own window with Mac menus, file pickers and downloads; stops the server on quit. A watchdog in the server exits if the shell dies.
Web UI	Vanilla JavaScript single-page app served by Spring (Thymeleaf templates + static assets). No framework or build step; fonts are bundled locally.
SecurityFilter	Rejects requests whose Host is not localhost (DNS-rebinding defence), checks Origin on writes, requires a signed-in session, guards admin routes, and adds CSP and other security headers.
Controllers	REST endpoints under /api (workspace), /api/auth (accounts) and /api/admin (dashboard), plus page routes.
Services	RequestExecutor sends HTTP requests (variables, auth, bodies, timeouts); TestRunner evaluates assertions and captures; ReqlyService owns workspace CRUD and import/export; AuthService handles accounts and throttling; AdminService powers the dashboard.
Persistence	Spring Data JPA repositories, every query scoped to the current user. Sensitive columns are encrypted with AES-256-GCM. Flyway versions the schema; a backup is taken before each migration and the database is compacted on shutdown.

Request flow (Send)

- The UI collects the request, runs the pre-request script in a sandboxed iframe, and posts it to `/api/execute`.
- The server resolves {{variables}} from the selected environment, applies auth, builds the body and sends it with Java's HttpClient.
- Assertions and captures run on the response; captured values are saved to the environment; history is recorded (headers encrypted).
- The UI renders the response, runs the post-response script and shows test results and console output.

Security model

- Server listens on 127.0.0.1 only; Host and Origin checks block other websites from driving it.
- Passwords hashed with BCrypt; sessions are random tokens stored only as SHA-256 hashes, sent in an HttpOnly, SameSite=Strict cookie; sign-in throttling (5 attempts per minute).
- Environment variables, request headers, auth configuration and history request headers are encrypted at rest (AES-256-GCM, key in `~/reqly-data/reqly.key`).
- User scripts run in a sandboxed iframe with no access to the app, cookies or storage; HTML previews render with scripts disabled.
- Admin SQL console is read-only (single SELECT/WITH/EXPLAIN/SHOW, row and time limits); hashes, tokens and encrypted values are always masked.

3. Tech stack

Area	Technology	Version / notes
Language (server)	Groovy on the JVM	Groovy 4.0.23, Java 17 target
Framework	Spring Boot (Web, Data JPA, Validation, Thymeleaf)	3.3.4
Persistence	Hibernate ORM via Spring Data JPA	managed by Spring Boot 3.3
Database	H2 embedded file database	default; MariaDB driver 3.3.3 available for a server profile
Migrations	Flyway	H2 migrations in `db/migration/h2`
Security	spring-security-crypto (BCrypt), javax.crypto AES-256-GCM	custom SecurityFilter, no Spring Security filter chain
HTTP client	java.net.http.HttpClient	Apache HttpClient 5.3.1 also on classpath
JSON / CSV	Jackson, groovy-json, OpenCSV	Jackson 2.16.1, OpenCSV 5.9
Front end	HTML, CSS, vanilla JavaScript	no framework or bundler; local web fonts (OFL)
Desktop app	Swift + AppKit + WKWebView shell; jpackage	macOS 12+, bundles its own Java runtime
Build	Gradle (wrapper)	tasks: bootRun, test, bootJar, packageMacApp, packageMac
Tests	JUnit 5, Spring MockMvc	38 unit and integration tests
CI	GitHub Actions	`.github/workflows/ci.yml` runs the test suite on push

Project layout

```
src/main/groovy/com/reqly/
  controller/   ApiController, AuthController, AdminController, PageController
  service/      ReqlyService, RequestExecutor, TestRunner, AuthService, AdminService,
                PostmanConverter, CurlParser, CsvDataReader
  entity/       ApiRequest, Collection, Environment, RequestHistory, AppUser, AuthSession
  repository/   Spring Data repositories (owner-scoped queries)
  security/     SecurityFilter, CurrentUser, EncryptedStringConverter, EncryptExistingSecrets
  config/       DatabaseBackup, DatabaseCompactor, PasswordResetRunner, JacksonConfig
  desktop/      DesktopLauncher
src/main/resources/
  templates/    index, login, admin, privacy, terms, licenses
  static/       js/ (app, scripting, terminal, login, admin), css/, fonts/
  db/migration/h2/   V1__baseline, V2__accounts, V3__admin_role
packaging/macos/     Reqly.icns, shell/main.swift, shell/Reqly.entitlements
```

4. Installation

Option A: macOS app (recommended)

- Open `Reqly-1.0.0.dmg` and drag **Reqly** into Applications. The app bundles its own Java runtime.
- Launch Reqly. It opens in its own window (not a browser) and starts its local server automatically.
- On first launch, create an account on the sign-in screen. The first account on a machine becomes the admin.
- Quit with Cmd+Q; the server stops and the database is compacted. Logs: **Help > Show Logs** (`~/reqly-data/logs`).
- Unsigned builds show an "unidentified developer" warning on other Macs: right-click the app and choose Open, or distribute a signed and notarised build.

Option B: run from source

Prerequisites: Java 17+, Git.

```
git clone https://github.com/ShaikRaihan93/reqly.git
cd reqly
./gradlew bootRun          # then open http://localhost:8899
```

UI files are served directly from `src/` during bootRun, so front-end edits show up on reload.

Building installers

```
./gradlew test                 # run the test suite
./gradlew packageMacApp        # build/dist/Reqly.app
./gradlew packageMac           # build/dist/Reqly-1.0.0.dmg
./gradlew packageMac -PmacSignIdentity="Developer ID Application: Name (TEAMID)
```

Packaging needs Xcode command-line tools (for swiftc). The app is built for the CPU of the JDK that runs Gradle; use an Apple-silicon (aarch64) JDK for a native arm64 app.

Data, backups and recovery

Path / command	Purpose
`~/reqly-data/reqly.mv.db`	H2 database file
`~/reqly-data/reqly.key`	Encryption key for secret columns. Back it up with the database; without it encrypted values cannot be read.
`~/reqly-data/backups/`	Automatic zip backups taken before schema migrations, plus manual ones from the admin dashboard
`~/reqly-data/logs/`	Server log (reqly.log) and shell console output
`java -jar reqly-1.0.0.jar --reqly.reset-password=you@example.com`	Resets a forgotten password from the command line (prints a temporary password)

5. Admin dashboard

The admin dashboard is a separate page for managing accounts and inspecting the database. Only admins can open it; for everyone else `/admin` and `/api/admin/**` are refused.

How to open it

- Sign in with an admin account. The first account created on a machine is admin automatically; admins can promote others.
- Click your **name / avatar in the top-right** corner to open the account menu, then choose **Admin dashboard**. In the desktop app it opens in its own "Reqly Admin" window.
- Or go directly to `http://localhost:8899/admin` while signed in.

What it offers

Section	Features
Overview	Row counts per table, database file and size, Flyway schema version, server info, latest backups
Users	Workspace size and active sessions per user; make / remove admin (at least one admin is kept); sign out everywhere; reset password (temporary password shown once); delete an account and its data (not your own)
Tables	Browse every table with search, sorting, paging, CSV export and a row-detail panel
SQL console	Read-only: one SELECT / WITH / EXPLAIN / SHOW statement, max 1,000 rows, 10-second timeout; writes and H2 file functions rejected
Backups	Create a backup now and download any existing backup

Password hashes, session tokens and encrypted values are always masked in the dashboard. To open the raw database in an external tool (DBeaver, DataGrip), quit Reqly first or open a copy from the backups folder: JDBC URL `jdbc:h2:file:~/reqly-data/reqly`, user `sa`, empty password.

6. Database schema

Six tables in an H2 file database, versioned by Flyway. Every workspace table carries an `owner_id` so each account only ever sees its own data. Columns marked **encrypted** are stored as AES-256-GCM ciphertext (prefix `enc:v1:`).

Relationships

```
app_users 1 --- * auth_sessions        (ON DELETE CASCADE)
app_users 1 --- * collections          (owner_id)
app_users 1 --- * api_requests         (owner_id)
app_users 1 --- * environments         (owner_id)
app_users 1 --- * request_history      (owner_id)
collections 1 --- * collections        (parent_id: nested folders)
collections 1 --- * api_requests       (collection_id)
environments 1 --- * api_requests      (environment_id)
api_requests 1 --- * request_history   (request_id)
```

app_users

Column	Type	Notes
id	BIGINT identity	Primary key
name	VARCHAR(255)	Display name
email	VARCHAR(255)	Unique (uk_app_users_email)
password_hash	VARCHAR(255)	BCrypt hash
admin	BOOLEAN	Default false; earliest account set to true (V3)
created_at, last_login_at	TIMESTAMP(6)	

auth_sessions

Column	Type	Notes
id	BIGINT identity	Primary key
token_hash	VARCHAR(64)	SHA-256 of the session token; unique
user_id	BIGINT	FK app_users(id), ON DELETE CASCADE
expires_at	TIMESTAMP(6)	Session expiry (30 days with "remember me")
created_at	TIMESTAMP(6)	

collections

Column	Type	Notes
id	BIGINT identity	Primary key
name	VARCHAR(255)	Required
description	VARCHAR(255)	
parent_id	BIGINT	FK collections(id): folder nesting
sort_order	INTEGER	Drag-and-drop order
owner_id	BIGINT	FK app_users(id), indexed
created_at, updated_at	TIMESTAMP(6)	

api_requests

Column	Type	Notes
id	BIGINT identity	Primary key
name	VARCHAR(255)	Required
method	VARCHAR(255)	GET, POST, ...
url	VARCHAR(2048)	Base URL (query kept in params)
params	VARCHAR	Query parameter rows (JSON)
headers	VARCHAR	Header rows (JSON), **encrypted**
disabled_items	VARCHAR	Disabled params / headers (JSON)
body, body_type	VARCHAR, VARCHAR(255)	none, raw, text, x-www-form-urlencoded, form-data, graphql
auth_type, auth_config	VARCHAR(255), VARCHAR	bearer / basic / apikey; config **encrypted**
settings	VARCHAR	Timeout, redirects, TLS (JSON)
test_script	VARCHAR	Line assertions and captures
pre_request_script, post_response_script	VARCHAR	pm-API JavaScript
collection_id	BIGINT	FK collections(id)
environment_id	BIGINT	FK environments(id)
sort_order	INTEGER	
owner_id	BIGINT	FK app_users(id), indexed
created_at, updated_at	TIMESTAMP(6)	

environments

Column	Type	Notes
id	BIGINT identity	Primary key
name	VARCHAR(255)	Required
variables	VARCHAR	Key/value map (JSON), **encrypted**
secrets	VARCHAR	Names of variables masked in the UI (JSON)
owner_id	BIGINT	FK app_users(id), indexed
created_at, updated_at	TIMESTAMP(6)	

request_history

Column	Type	Notes
id	BIGINT identity	Primary key
method, url	VARCHAR(255), VARCHAR(2048)	What was sent
request_headers	VARCHAR	**encrypted**
request_body	VARCHAR	
status_code	INTEGER	
response_headers, response_body	VARCHAR	
response_time_ms, response_size_bytes	BIGINT	
test_results	VARCHAR	Assertion results (JSON)
request_id	BIGINT	FK api_requests(id)
owner_id	BIGINT	FK app_users(id); index (owner_id, executed_at)
executed_at	TIMESTAMP(6)	

Migration history

Version	File	Change
V1	`V1__baseline.sql`	Pre-accounts schema: collections, api_requests, environments, request_history. Existing installs are baselined here.
V2	`V2__accounts.sql`	Adds app_users and auth_sessions, plus owner_id columns, foreign keys and indexes on all workspace tables.
V3	`V3__admin_role.sql`	Adds app_users.admin and makes the earliest account the first admin.

7. REST API summary

All endpoints require a signed-in session except sign-up, sign-in and status. Admin endpoints require an admin account.

Area	Endpoints
Accounts	`GET /api/auth/status, /me` `POST /api/auth/signup, /signin, /signout, /password, /profile`

Workspace	`GET /api/tree`
	`GET|POST /api/collections, PATCH|DELETE /api/collections/{id}`
	`POST /api/collections/{id}/duplicate, GET .../export, POST /api/collections/reorder`
	`GET|POST /api/requests, GET|PATCH|DELETE /api/requests/{id}`
	`POST /api/requests/{id}/duplicate, GET .../history, POST /api/requests/reorder`

Execution	`POST /api/execute, /api/execute/multipart, /api/execute/bulk, /api/execute/csv`
Environments / history	`GET|POST /api/environments, DELETE /api/environments/{id}` `GET|DELETE /api/history`
Import / export	`POST /api/import/curl, /api/import/postman, /api/import/all` `GET /api/export/all`

Admin	`GET /api/admin/overview, /tables, /tables/{name}, /users, /backups`
	`POST /api/admin/query, /users/{id}/admin, /users/{id}/signout, /users/{id}/reset-password, /backups`
	`DELETE /api/admin/users/{id}, GET /api/admin/backups/{name}`

Pages	`/ /login /admin /privacy /terms /licenses`

8. Future enhancements

Proposed roadmap, grouped by theme. Items are suggestions for prioritisation, not commitments.

Platform and distribution

- **Windows and Linux desktop apps**: the same native-shell approach using WebView2 (Windows) and WebKitGTK (Linux), with MSI and AppImage/DEB installers.
- **Apple-silicon native build and notarisation**: arm64 JDK in CI, Developer ID signing, notarised DMG and an auto-update channel (e.g. Sparkle).
- **Faster start-up**: class-data sharing / AOT cache and lazy initialisation to cut the server start time.

Collaboration and sync

- **Team workspaces**: optional self-hosted server mode on MariaDB/PostgreSQL with shared collections, roles and an audit log.
- **Git-friendly storage**: export collections as readable files so they can be versioned and reviewed alongside code.
- **End-to-end encrypted sync** between a user's own devices.

Protocols and requests

- WebSocket, Server-Sent Events and gRPC requests; GraphQL schema introspection and autocomplete.
- OAuth 2.0 flows (authorization code with PKCE, client credentials) with automatic token refresh; AWS SigV4 and digest auth.
- Client certificates (mTLS), per-request proxy settings and cookie jar management.

Testing and automation

- **Command-line runner** (like Newman) to run collections in CI with JUnit/HTML reports.
- Scheduled monitors with alerting, response-time trends and contract tests against OpenAPI schemas.
- Mock servers generated from saved examples.

Import, docs and ecosystem

- OpenAPI / Swagger and Insomnia import; publish collections as API documentation.
- More code-snippet languages (Go, C#, PHP, Kotlin) and a plugin API for custom auth and scripts.

Product and commercial readiness

- Licensing and activation, in-app onboarding tour, crash reporting with opt-in telemetry.
- Final legal review of the privacy policy and terms (current versions are templates), accessibility audit (WCAG 2.2 AA), localisation.
- Password recovery by email for the self-hosted team mode, plus optional two-factor sign-in.